

Module 12 – Reinforcement Learning

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Data Science	M12-A1	2 Hours	Python 3

GENERAL INSTRUCTIONS

Attempt all sections. Write your code in Python 3 unless stated otherwise. For coding tasks, submit your .py source file along with a screenshot of your program output. Label every task clearly with its section and task number. All work must be your own.

A Section A – Concept Application

Answer all questions. Each question presents a real-world scenario followed by a concept-specific question.

SCENARIO

**S
1**

You are a data scientist at a food delivery company building an AI system that must automatically improve its delivery dispatching decisions over time, without being given any labelled examples of correct or incorrect dispatch choices.

Question: Explain why Reinforcement Learning is more suitable than traditional programming for this dispatch system. What key characteristic of RL allows it to improve its decisions without relying on labelled training data?

SCENARIO

**S
2**

You are building a restaurant recommendation engine and your manager suggests training a supervised learning model on historical order data. A colleague argues that a Reinforcement Learning agent would produce better results over time.

Question: Compare how a supervised learning model and a Reinforcement Learning agent each learn to make recommendations. What fundamental difference in their learning mechanism would justify choosing RL for a continuously improving recommendation engine?

SCENARIO

S
3

You are evaluating whether to use unsupervised learning (clustering) or Reinforcement Learning to optimise the delivery zones assigned to riders on your platform. Both approaches work without labelled data, yet your technical lead says they are fundamentally different.

Question: What distinguishes a Reinforcement Learning system from an unsupervised learning system when the goal is to continuously improve delivery zone assignments based on real delivery outcomes?

SCENARIO

S
4

You are designing an RL agent for your food delivery platform that must decide which rider to assign to each incoming order. Your team needs to define the core components of the RL system before implementation begins.

Question: Identify the agent, environment, set of possible actions, and reward signal for this delivery dispatch RL system. Justify why your chosen reward signal directly encourages efficient delivery performance rather than unintended shortcut behaviour.

SCENARIO

S
5

You are presenting Reinforcement Learning to your company's product team, who are familiar with how game-playing AI (such as chess engines) works. They want to understand how the same principle applies to food delivery.

Question: Using game AI as a reference point, explain the core principle of how an RL agent learns through interaction with its environment. In what ways does the delivery dispatch scenario follow this same learning principle even though it is not a game?

SCENARIO

S
6

You are advising your company on adopting an RL-based system to personalise meal suggestions for each customer on your platform, similar to how streaming services use RL to personalise content recommendations.

Question: Describe how Reinforcement Learning has been applied to recommendation systems in industry. What advantage does an RL-based recommendation system offer over a model trained once on historical data, particularly for customers whose food preferences change over time?

B

Section B — Practical Coding Tasks

Complete all tasks. Each task targets one specific concept or feature from the module. Follow the requirements listed under each task.

Task 1: Delivery Reward Function

Build a Python function that calculates and returns a reward score for a food delivery agent based on the outcome of a single delivery action.

- Define a function `calculate_reward(delivery_time, is_on_time, customer_rating)` that accepts three parameters.
- Award +10 points if `is_on_time` is True; deduct -5 points if it is False.
- Add the `customer_rating` (integer 1–5) directly to the reward total.
- Print the final reward with a descriptive message and test the function with at least three different input combinations showing different outcomes.

Task 2: Agent State Tracker

Build a dictionary-based Python program that tracks a delivery agent's state across multiple time steps, simulating the state component of an RL environment.

- Create a dictionary representing the agent's state with keys: `location` (string), `orders_delivered` (integer), `total_reward` (float), and `is_available` (boolean).
- Define a function `update_state(state, new_location, reward_earned)` that updates the agent's location, increments `orders_delivered` by 1, and adds `reward_earned` to `total_reward`.
- Simulate at least four state transitions by calling `update_state` with different locations and reward values.
- After each update, print the complete current state in a clear, readable format.

Task 3: Action-Reward Episode Logger

Build a Python program that simulates one episode of a delivery RL agent executing a fixed sequence of actions and logs the cumulative reward at every step.

- Define a list of possible actions: `['accept_order', 'reject_order', 'request_directions', 'mark_delivered']`.
- Assign a fixed reward to each action using a dictionary: `accept_order = +2`, `reject_order = -1`, `request_directions = 0`, `mark_delivered = +10`.
- Simulate an episode by iterating through a predefined sequence of at least six actions, computing and accumulating the reward at each step.

- Print each step showing the action taken, the reward received at that step, and the running cumulative reward. Print the total episode reward at the end.

Task 4: Simple RL Environment Simulator

Build a console-based Python simulation that models the full agent-environment interaction loop for a delivery scenario, combining state tracking, action selection, and reward accumulation into a single class-based program.

- Define a `DeliveryEnvironment` class with attributes: `current_state` (a dictionary with at least `location` and `pending_orders`), `total_reward`, and `step_count`.
- Implement a `step(action)` method that updates the state based on the action taken, computes and returns a reward value, and prints the new state and reward after each step.
- Define at least three distinct actions with different state-transition effects and reward values (e.g. `deliver_order`, `wait`, `navigate_to_zone`).
- Run a loop that executes a predefined sequence of five or more actions, prints the result of each step, and displays the final total reward and total step count at the end.

C

Section C — Mini Capstone Project

Mini Project: Food Delivery RL Concept Simulator

Objective:

Build a console-based Food Delivery Reinforcement Learning Simulator that models the full RL loop — agent, environment, actions, states, and rewards — for a delivery dispatch scenario, combining the reward function, state tracking, episode logging, and environment interaction concepts from Section B into a single working program.

Your project must:

- The program must present a menu-driven interface with at least three options: Start New Delivery Episode, View Agent Stats, and Exit.
- Each episode must simulate a delivery agent completing a sequence of at least five actions, with the agent's state updated and reward calculated after every action.
- The program must track and display cumulative statistics across multiple episodes: total episodes run, total reward earned, and average reward per episode.
- The agent's state (current location, orders delivered, total reward) must be stored in a dictionary and updated correctly after every action in every episode.
- At the end of each episode, the program must print a full episode summary showing each action taken, the reward received per step, and the total episode reward.

D

Section D — AI-Augmented Learning

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a program that:

- Simulates a simple food delivery RL episode where an agent takes a sequence of actions and accumulates rewards.
- Tracks the agent's state (location, reward, step count) and updates it correctly after every action.
- Prints a per-step log showing the action taken, the reward received at that step, and the running total reward.
- Prints a final episode summary including total reward and total steps taken.

STEP 2 • TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the code and find at least one bug, limitation, or improvement in the AI's solution. Fix it yourself.

SUBMIT

All 3 items required

- 1 The exact prompt(s) you gave the AI tool.
- 2 The AI's original code and your corrected version.
- 3 A 3–4 line note explaining what you changed and why the AI's version needed it.